

Sessions — Shopify Storefront Event Collector

A lightweight Node.js/Express service that collects visitor behavior events (page views, product views, add-to-cart, checkout starts, scroll depth, and clicks) from Shopify storefronts, stitches them into visitor sessions, and stores everything in MongoDB.

Note on the name: despite the folder being called "sessions," this is **not** a booking/appointment-scheduling app. "Session" here means a *website visit session* — the standard web-analytics concept of one continuous browsing visit by a single person.

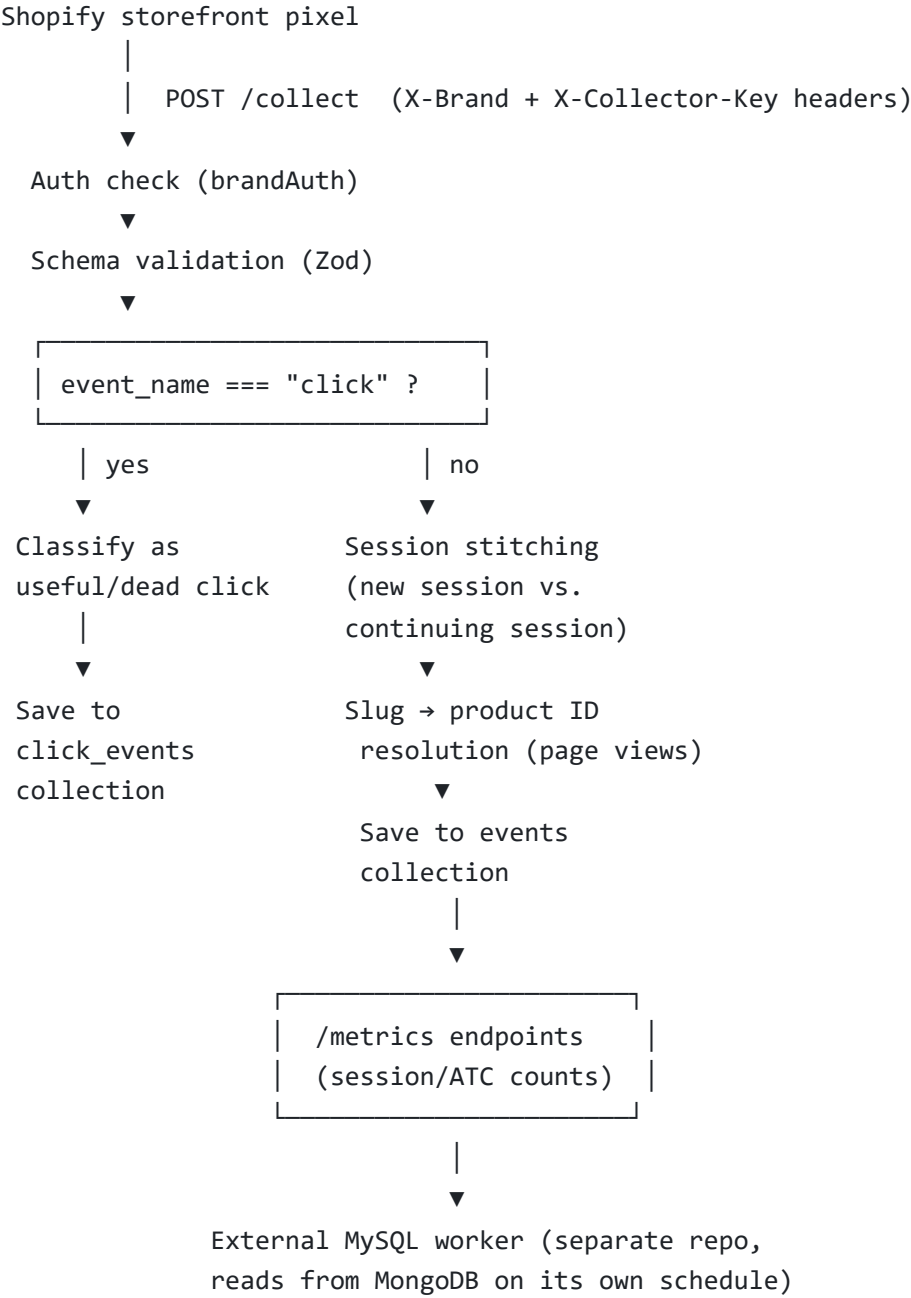
1. What this service does

Shopify stores install a small tracking script (a "pixel") that watches what a visitor does — viewing pages, viewing products, adding items to cart, starting checkout, scrolling, and clicking. Every time one of those things happens, the pixel sends a small JSON message to this service's `/collect` endpoint. This service:

1. Verifies the request belongs to a known store ("brand").
2. Validates the shape of the event.
3. Figures out which ongoing visit ("session") the event belongs to.
4. For clicks, classifies whether it was a **useful** click (it did something) or a **dead** click (it did nothing).
5. Saves the event to MongoDB, avoiding duplicates.

A separate, external service (in a different repository, not this one) periodically reads this data out of MongoDB and loads it into a MySQL reporting table. That worker is out of scope for this repo — this service's job ends at "reliably capture and store events in MongoDB."

2. High-level flow



3. Event types tracked

Event name	What it captures	Special handling
page_viewed	A storefront page load	The URL is parsed for a product/collection slug, which is looked up in a slug cache to attach a real Shopify product ID
product_viewed	A visitor viewing a specific product	Product/variant details captured as-is

Event name	What it captures	Special handling
<code>product_added_to_cart</code>	An add-to-cart action	Deduplicated per (brand, session, product) so repeat pixel fires for the same item don't double-count; the only event type aggregated by <code>/metrics/sessions/:timestamp</code>
<code>checkout_started</code>	A checkout beginning (native Shopify checkout or a third-party checkout accelerator like GoKwik)	Both checkout paths are normalized to the same <code>checkout_started</code> event name, with a <code>source</code> field noting which one fired
<code>scroll_depth</code>	A visitor scrolling past a threshold (25%/50%/75%/100%) on a page	Published as a custom event from the storefront theme, forwarded by the pixel
<code>click</code>	Any click on the page	The pixel watches what happens <i>after</i> the click and reports four true/false signals; the server uses those to classify the click (see below)

Any other event name is still accepted and stored generically (matched only by `event_id` for deduplication) — the server doesn't require a fixed list of event names.

4. Data model (MongoDB collections)

Collection	Purpose	Retention (TTL)
<code>sessions</code>	One document per visitor session — landing page, referrer, UTM parameters, first/last activity timestamps	36 hours
<code>events</code>	All non-click events (page views, product views, add-to-cart, checkout starts, scroll depth)	45 minutes
<code>click_events</code>	Every click event, with its classification (<code>useful_click</code> / <code>dead_click</code>)	45 minutes

Collection	Purpose	Retention (TTL)
<code>slug_cache</code>	Maps a store's URL slugs to real Shopify product IDs, used to enrich <code>page_viewed</code> events	—
<code>slug_queue</code>	A queue of slugs awaiting resolution — written for, and read by, the external slug-resolution worker (not part of this repo; currently unused by <code>server.js</code> itself)	—

Retention windows are intentionally short (45 min / 36h) because this service is a fast-moving ingestion buffer — the external MySQL worker is expected to read events out well before they expire.

Every event is written with an idempotent "insert if not already present" pattern, keyed on the event's own `event_id`. That means if the same event is sent twice (a network retry, a double pixel fire), it's stored once, not duplicated. `product_added_to_cart` events additionally dedupe on `(brand, session, product)`, so the same product can't be double-counted as added-to-cart within the same session.

5. Useful click vs. dead click

Not every click does something — some just miss their target, or hit dead space. To separate signal from noise, the pixel watches for four outcomes after a click:

- `url_changed` — did the page navigate?
- `cart_changed` — did the cart contents change?
- `ui_changed` — did something visible on the page change (a modal opened, a menu expanded, etc.)?
- `meaningful_scroll` — did a real scroll happen as a result (e.g. jumping to a different section)?

The pixel computes these four booleans itself and sends them along with the click. The server's job is simple: if **any** of the four are `true`, the click is classified `useful_click`; if **all** are `false`, it's a `dead_click`. This classification is stored directly on the click document for later analysis (e.g. "which buttons get clicked but do nothing").

6. Session stitching, in plain terms

A "session" groups together everything one visitor did in one continuous visit. The rules:

- If a visitor hasn't been seen in the last **30 minutes**, their next event starts a **new** session.
- If they're within that 30-minute window, the event is added to their **existing** session.
- **Exception:** even within 30 minutes, if a page view arrives with a meaningfully different traffic source (e.g. they left, came back from a different ad campaign or a different platform like Google vs. Facebook), the session is **split** — a new session starts, so campaign attribution doesn't get muddled by re-entries. This split rule is debounced (won't fire on rapid, likely-accidental source hops) and only considered on page views, not every event type.

7. API endpoints

Endpoint	Method	Purpose
<code>/collect</code>	POST	Main ingestion endpoint. Requires <code>X-Brand</code> and <code>X-Collector-Key</code> headers (or <code>?brand=</code> / <code>?key=</code> query params). Accepts one event JSON body per call. Returns <code>204</code> on success, <code>400</code> on validation failure, <code>401</code> on bad auth.
<code>/metrics/sessions</code>	GET	Returns a session count for a brand within a time range (<code>?from=</code> / <code>?to=</code> , defaults to the last 24h).
<code>/metrics/sessions/:timestamp</code>	GET	Returns session count and unique add-to-cart-session count for a brand, from a given timestamp to now (or <code>?to=</code>).
<code>/healthz</code>	GET	Basic health check, no auth required.

8. Authentication

Every brand (Shopify store) using this pipeline has its own secret key. A request must include:

- `X-Brand` : the store's identifier (e.g. `pts_shop`)
- `X-Collector-Key` : that store's secret key

These are checked against the `COLLECTOR_KEYS` environment variable, a JSON map of `brand` → `key` . If a brand isn't in that map, requests fall back to checking against a single shared `COLLECTOR_KEY` value instead — but in practice, every active brand should have its own entry in `COLLECTOR_KEYS` .

9. Environment variables

Variable	Used for
<code>MONGO_URI</code>	MongoDB connection string, including the database name (important: without an explicit database name in the URI, Mongo silently defaults to a database literally called <code>test</code> , which can collide with unrelated data — always include a dedicated database name)
<code>PORT</code>	Port the server listens on
<code>COLLECTOR_KEY</code>	Single fallback auth key, used only for brands not listed in <code>COLLECTOR_KEYS</code>
<code>COLLECTOR_KEYS</code>	JSON map of <code>brand</code> → <code>secret key</code> , the primary per-store authentication mechanism
<code>SHOPIFY_SHOPS</code>	JSON map of <code>brand</code> → Shopify store domain/API token — present in <code>.env</code> but not currently read anywhere in <code>server.js</code> ; reserved for the external slug-resolution/MySQL worker
<code>WORKER_ENABLED</code>	Present in <code>.env</code> but not currently read anywhere in <code>server.js</code> ; reserved for the external worker
<code>DEBUG_SESSIONS</code>	Present in <code>.env</code> ; currently unused by application logic

10. Running locally & deploying

```
npm install
npm run dev      # nodemon, auto-restarts on file changes
# or
npm start        # plain node
```

Both commands load environment variables from a local `.env` file via Node's built-in `--env-file` flag — no `dotenv` package is used.

Important deployment note: if this service is deployed on a platform like Render, that platform's environment variables are configured **separately in its own dashboard** — they are not automatically synced from the local `.env` file (which is git-ignored and never pushed). When updating secrets like `COLLECTOR_KEYS` or `MONGO_URI`, both places need to be updated in lockstep, or the deployed service will silently keep using stale values. This has been a real source of confusing bugs (e.g. a brand's key working locally but returning `401` in production).

Also note: Render's dashboard treats environment variable values literally — unlike a `.env` file, it does **not** strip surrounding quote characters. Pasting a value that was copied straight out of a `.env` file (which may be wrapped in `'...'` for shell-quoting purposes) will include those quote characters in the actual value, breaking JSON parsing for variables like `COLLECTOR_KEYS`.

11. Things to know when working on this service

- **Deduplication is by `event_id`**, not by content — sending the exact same event twice with the same ID is safe; sending a logically duplicate event with a new ID will be stored twice.
- **Data doesn't live long here on purpose** — 45 minutes for events/clicks, 36 hours for sessions. This service is a buffer, not a permanent data store; downstream systems (the external MySQL worker) are expected to read data out promptly.
- **Click events skip session stitching** — unlike other events, a click's `session_id` is taken as-is from whatever the pixel already computed, without the server re-deriving/updating session state. This keeps high-volume click tracking lightweight.
- **The MySQL ingestion worker lives in a different repository** and is not part of this codebase — `slug_queue` and the `SHOPIFY_SHOPS / WORKER_ENABLED` env vars exist here only because this pipeline produces the data that worker consumes.
- **The Shopify pixel script itself is also maintained separately** (in the relevant Shopify store's admin, not in this repo) — this README documents the server side only.